

FITBUDDY

AI Fitness Plan Generator using Gemini Models

SAMPLE PROJECT ANALYSIS & DOCUMENTATION

A structured academic/project documentation based on the uploaded FitBuddy project document and the supplied sample documentation format.

Technology Stack: FastAPI • Google Gemini • SQLAlchemy • SQLite • Jinja2 • HTML/CSS

1. Project Overview

FitBuddy is a web-based application that uses Generative AI to generate personalized workout plans and nutrition tips based on a user's fitness goals. The documented implementation uses FastAPI as the backend framework, Google Gemini models as the AI layer, SQLite with SQLAlchemy for persistence, and Jinja2 templates for the web interface.

The application accepts user information such as name, age, weight, fitness goal, and preferred workout intensity, then generates a customized 7-day fitness plan. Users can also submit feedback so the AI can revise the existing plan. A complementary nutrition or recovery tip is generated for the selected goal.

Source basis: FitBuddy project documentation. ■filecite■turn0file0■L3-L13■

2. Problem Statement

Traditional fitness planning can require manual selection of exercises, scheduling, progression, and nutrition guidance. FitBuddy addresses this by combining a simple web form with Generative AI so that a structured fitness routine can be generated from a small set of user inputs.

The system also addresses changing user preferences through a feedback loop: the original plan is retained, feedback is supplied to the AI, and an updated plan is stored separately.

3. Objectives

- Generate structured 7-day workout plans from user fitness inputs.
- Provide concise nutrition or recovery guidance aligned with the selected goal.
- Allow users to refine plans through natural-language feedback.
- Store user information and generated plans using SQLite and SQLAlchemy.
- Provide API endpoints and a browser-based HTML interface.
- Provide an administrative view for registered users and their original/updated plans.

4. Main Use Cases / Scenarios

Scenario 1 – Personalized Plan

The user enters personal details, fitness goal, and workout intensity. FitBuddy generates a day-by-day 7-day workout schedule tailored to the objective and experience level.

Scenario 2 – Feedback-Based Update

The user submits feedback such as adding more cardio or rest days. Gemini processes the original plan plus the feedback and returns a revised plan.

Scenario 3 – Nutrition / Recovery Tip

The user receives a concise tip relevant to the selected goal, such as protein or hydration guidance.

Scenario 4 – Admin / Coach View

An administrator or coach can view registered users and their workout plans, including original and updated versions.

These scenarios are directly documented in the uploaded FitBuddy specification. ■filecite■turn0file0■L14-L31■

5. Technology Stack Analysis

Layer	Technology	Role in FitBuddy
Frontend	HTML + CSS + Jinja2	User input, result display, feedback, and admin pages
Backend	FastAPI	Routing, form processing, validation, API handling, and orchestration
AI Layer	Google Gemini Pro + Flash	Workout generation, plan updates, and fast nutrition/recovery tips
Database	SQLite + SQLAlchemy	Persistent storage for users and generated plans
Templating	Jinja2	Dynamic server-rendered HTML
Server	Uvicorn	Runs the FastAPI application locally
Language	Python	Application logic and AI integration

The source documentation explicitly describes a modular FastAPI architecture with HTML/Jinja2 frontend, FastAPI backend, Gemini AI APIs, and SQLite/SQLAlchemy persistence. [filecite turn0file0 L97-L114](#)

6. AI Model Analysis

The project documentation reports that Hugging Face models such as T5, BART, DistilBERT, and BLOOM were initially explored. It identifies hosting/local GPU requirements and additional prompting or fine-tuning as limitations for the intended structured-plan workflow.

The documented production selection is Gemini 1.5 Pro for structured 7-day workout generation and plan updating, and Gemini Flash for fast nutrition tips. The stated reasons include API integration with FastAPI, structured formatting, low prompt-tuning requirements, and speed for feedback-based revisions.

Source: FitBuddy model-selection section. [filecite turn0file0 L67-L96](#)

7. System Architecture

FitBuddy can be understood as a four-layer flow:

- Presentation Layer → HTML/Jinja2 templates collect inputs and display generated results.
- Application Layer → FastAPI routes receive and validate form data and coordinate the workflow.
- AI Layer → Gemini Pro generates/revises workout plans; Gemini Flash generates nutrition/recovery tips.
- Persistence Layer → SQLAlchemy communicates with SQLite to store user data and plans.

Core request flow:

User → index.html → FastAPI route → Gemini API → generated plan/tip → SQLite storage → result.html

The documented AI integration points are workout generation and feedback updates through Gemini Pro, and nutrition tips through Gemini Flash. [filecite turn0file0 L97-L114](#)

8. Core Modules

gemini_generator.py

`generate_workout_gemini()` — Generates a personalized 7-day plan containing warm-up, main workout, and cooldown/recovery guidance.

gemini_flash_generator.py

`generate_nutrition_tip_with_flash()` — Generates a concise nutrition or recovery tip.

updated_plan.py

update_workout_plan() — Uses the original plan plus user feedback to produce a revised plan.

database.py

save_user(), save_plan(), update_plan(), get_original_plan(), get_user() — Stores and retrieves user and plan records.

routes.py

FastAPI route handlers — Connects frontend inputs, AI functions, and database operations.

The documented implementation describes these functions and files across the core functionality milestone.

filecite[turn0file0]L129-L166

9. API / Route Analysis

Route	Method / Role	Functionality
/	GET	Displays index.html home/input page
/generate-workout	POST	Accepts user input; generates workout + nutrition tip; saves records
/submit-feedback	POST	Retrieves original plan, applies feedback through Gemini, saves updated plan
/view-all-users	GET	Displays users and original/updated plans for admin view

The route definitions and their responsibilities are documented in the uploaded FitBuddy file.

filecite[turn0file0]L199-L245

10. Database Design

The documented implementation uses SQLite through SQLAlchemy ORM. User information includes ID, name, age, weight, goal, and intensity. AI-generated plans are stored, while feedback-based updates are maintained separately so the original and updated versions can both be preserved.

Source: database/storage section. filecite[turn0file0]L158-L166

Sample ER-style conceptual model:

USER	WORKOUT_PLAN
user_id (PK)	plan_id (PK)
name	user_id (FK)
age	original_plan
weight	updated_plan
goal	feedback
intensity	nutrition_tip

Note: The uploaded project document describes the stored fields and separation of original/updated plans, but it does not provide a formal SQL schema or visual ER diagram. The diagram above is therefore a conceptual representation derived from the documented storage description, not a verbatim source artifact.

11. Frontend Analysis

Three Jinja2-powered templates are documented: index.html for input, result.html for the generated plan and feedback, and all_users.html for the admin panel. The UI uses a fitness-themed background, Google Fonts, Flexbox, responsive media queries, buttons with hover effects, and padded result blocks.

The result page displays user information, a day-wise workout plan, a nutrition tip, and a feedback form. The admin page displays user details and original/updated workout plans.

Source: frontend development section. ■filecite■turn0file0■L258-L305■

12. Development Workflow / Milestones

Milestone 1: Model Selection and Architecture

Research Gemini models, define architecture, and set up the development environment.

Milestone 2: Core Functionalities Development

Build workout generation, nutrition tips, feedback updates, and FastAPI backend processing.

Milestone 3: routes.py Development

Implement the main routing and operational application logic.

Milestone 4: Frontend Development

Develop HTML/CSS/Jinja2 pages and dynamic templates.

Milestone 5: Deployment

Prepare local deployment, configure environment variables, and verify the local server.

The five milestone structure is explicitly documented in the uploaded project file. ■filecite■turn0file0■L43-L60■

13. Installation & Run Guide

The source document specifies Python/pip, a virtual environment, FastAPI, Uvicorn, Jinja2, SQLAlchemy, SQLite basics, and the Google Generative AI SDK as development prerequisites.

Install dependencies:

```
python -m venv fitbuddy-env fitbuddy-env\Scripts\activate pip install fastapi uvicorn jinja2 sqlalchemy python-multipart google-generativeai
```

Run the application:

```
uvicorn app.main:app --reload
```

The documented local URLs are the application at http://127.0.0.1:8000 and FastAPI interactive documentation at /docs. ■filecite■turn0file0■L116-L127■

14. Deployment & Configuration

The project documentation recommends a virtual environment and requirements.txt for dependency management. Sensitive Gemini credentials are placed in environment variables or a .env file rather than hard-coded into application code.

Example configuration:

```
GOOGLE_API_KEY=your_gemini_api_key_here
```

The documented deployment section also describes running the FastAPI application with Uvicorn and verifying it through the local browser and /docs endpoint. ■filecite■turn0file0■L313-L339■

15. Functional Output Analysis

Workout output is described as day-wise and goal-oriented, with exercise names, sets/repetitions or duration, rest intervals, and optional warm-up/cool-down guidance. The nutrition section provides concise dietary advice, while the feedback feature allows the plan to evolve based on user input.

The documented web pages include a home page, personalized workout page, nutrition tip section, feedback page, and view-all-users page. ■filecite■turn0file0■L343-L399■

16. Strengths of the Documented Design

- Clear separation between frontend, backend, AI services, and persistence.
- Dedicated AI functions for workout generation, nutrition tips, and feedback updates.
- Preservation of original and updated plans supports traceability.
- FastAPI provides both browser-facing routes and interactive API documentation.
- Jinja2 enables server-side dynamic rendering with a relatively simple architecture.
- Gemini model selection is separated by workload: structured planning vs. fast tips.

17. Areas to Consider for a Real-World Extension

The uploaded document focuses on the project implementation and does not document several production concerns in depth. If this sample is extended beyond an academic/local project, the following can be treated as future engineering work rather than existing functionality: authentication/authorization, stronger privacy controls for personal data, input/output safety checks, fitness-risk screening, audit logging, automated tests, rate limiting, production database configuration, and cloud deployment.

These are recommendations for extension, not claims that these capabilities are already implemented in the supplied FitBuddy document.

18. Conclusion

FitBuddy is documented as an AI-powered fitness planning web application that combines FastAPI, Gemini models, SQLite/SQLAlchemy, and Jinja2. Its central workflow is personalized 7-day workout generation, supported by nutrition/recovery tips and a feedback-driven plan update mechanism.

The architecture is modular: the frontend collects user inputs, FastAPI manages application flow, Gemini provides generative intelligence, and SQLite/SQLAlchemy stores user and plan data. The project also includes an administrative view for reviewing users and plan versions.

The uploaded source concludes that FitBuddy demonstrates how Generative AI can be integrated into a wellness application to provide personalized, adaptable, and goal-oriented fitness support. ■filecite■turn0file0■L401-L420■

19. Source & Scope Note

Primary source: uploaded document “FitBuddy – AI Fitness Plan Generator using Gemini Models”. The supplied “Sample Project Documentation” was used as a formatting/organization reference; its content is not treated as FitBuddy functionality. The technical facts in this PDF are grounded primarily in the FitBuddy source. Where this PDF adds a conceptual diagram or future-extension discussion, it is explicitly identified as derived/recommended content.